Course Discussions

Solution: Problem Challenge 2: Search in Rotated Array

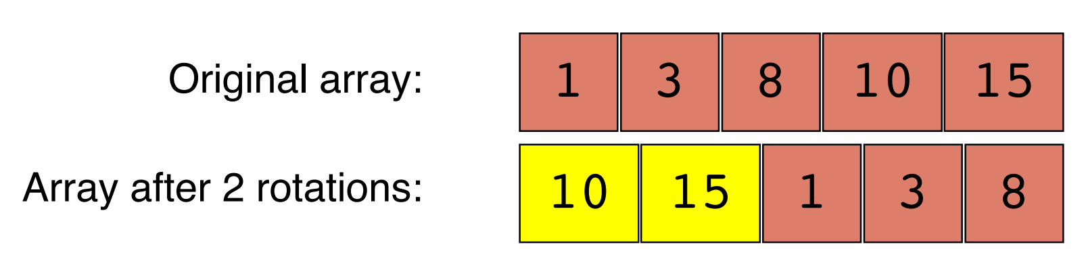
Problem Statement

Given an array of numbers which is sorted in ascending order and also rotated by some arbitrary number, find if a given 'key' is present in it.

Write a function to return the index of the 'key' in the rotated array. If the 'key' is not present, return -1. You can assume that the given array does not have any duplicates.

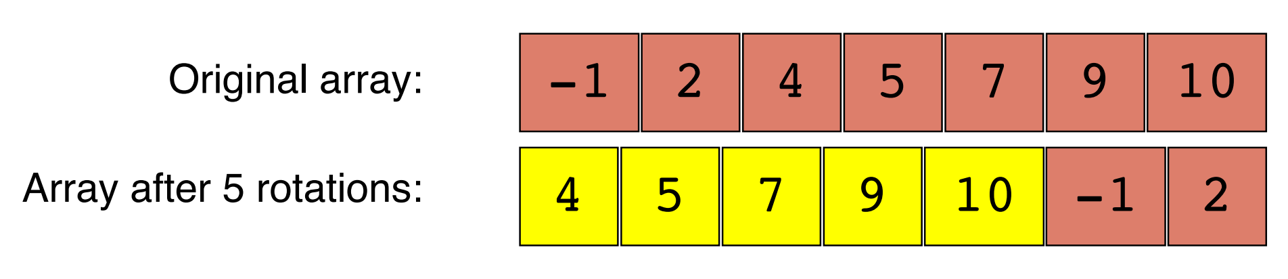
Example 1:

```
Input: [10, 15, 1, 3, 8], key = 15
Output: 1
Explanation: '15' is present in the array at index '1'.
```



Example 2:

```
Input: [4, 5, 7, 9, 10, -1, 2], key = 10
Output: 4
Explanation: '10' is present in the array at index '4'.
```



Solution

The problem follows the **Binary Search pattern.** We can use a similar approach as discussed in `Order-agnostic Binary Search` and modify it similar to `Search Bitonic Array` to search for the 'key' in the rotated array.

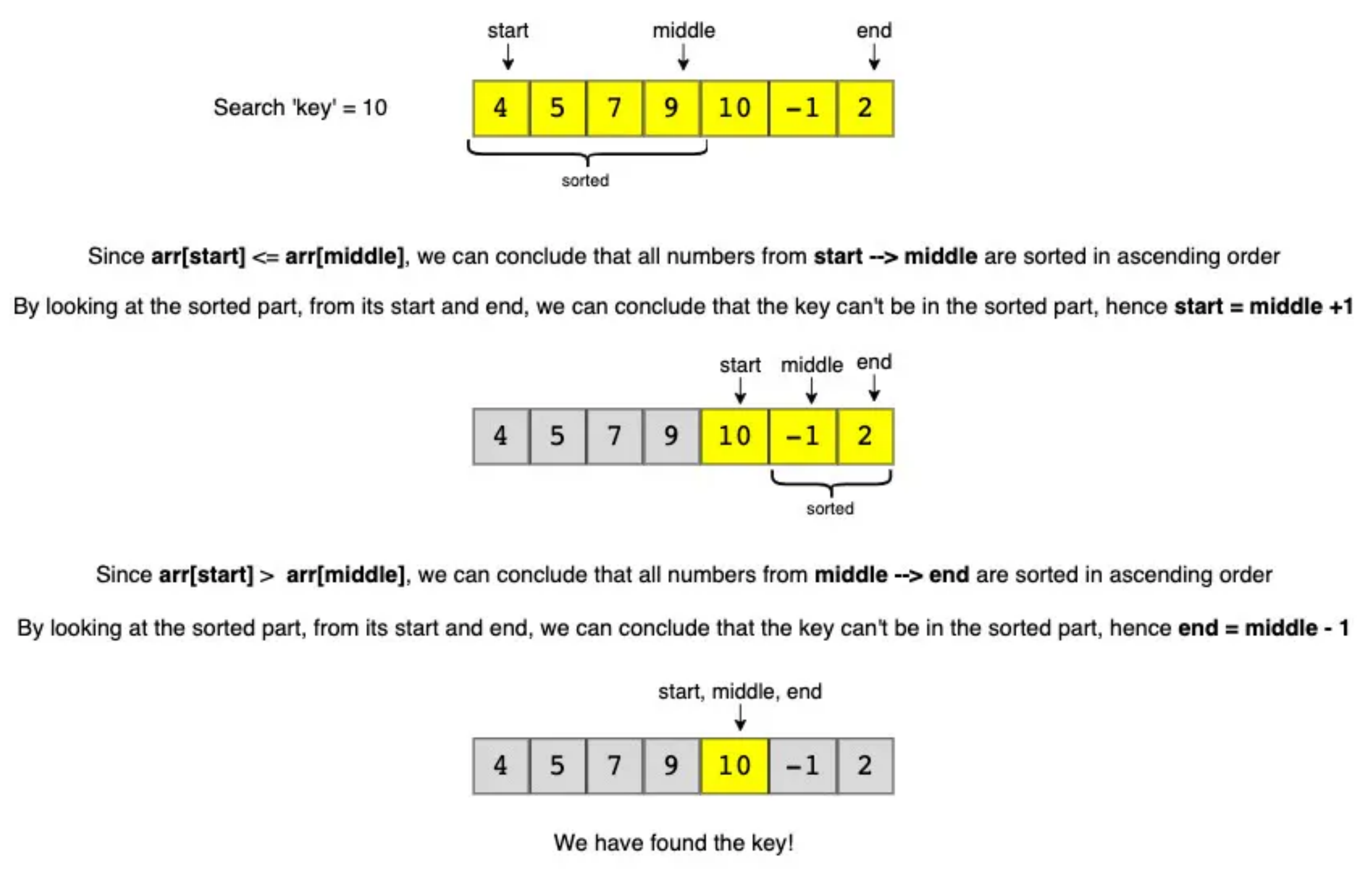
After calculating the `middle`, we can compare the numbers at indices `start` and `middle`. This will give us two options:

1. If `arr[start] <= arr[middle]`, the numbers from `start` to `middle` are sorted in ascending order.
2. Else, the numbers from `middle+1` to end are sorted in ascending order.

Once we know which part of the array is sorted, it is easy to adjust our ranges. For example, if option-1 is true, we have two choices:

1. By comparing the 'key' with the numbers at index `start` and `middle` we can easily find out if the 'key' lies between indices `start` and `middle`; if it does, we can skip the second part => `end = middle -1`.
2. Else, we can skip the first part => `start = middle + 1`.

Let's visually see this with the above-mentioned Example-2:



Since **arr[start]** <= **arr[middle]**, we can conclude that all numbers from **start --> middle** are sorted in ascending order

By looking at the sorted part, from its start and end, we can conclude that the key can't be in the sorted part, hence **start = middle +1**

Since **arr[start]** > **arr[middle]**, we can conclude that all numbers from **middle --> end** are sorted in ascending order

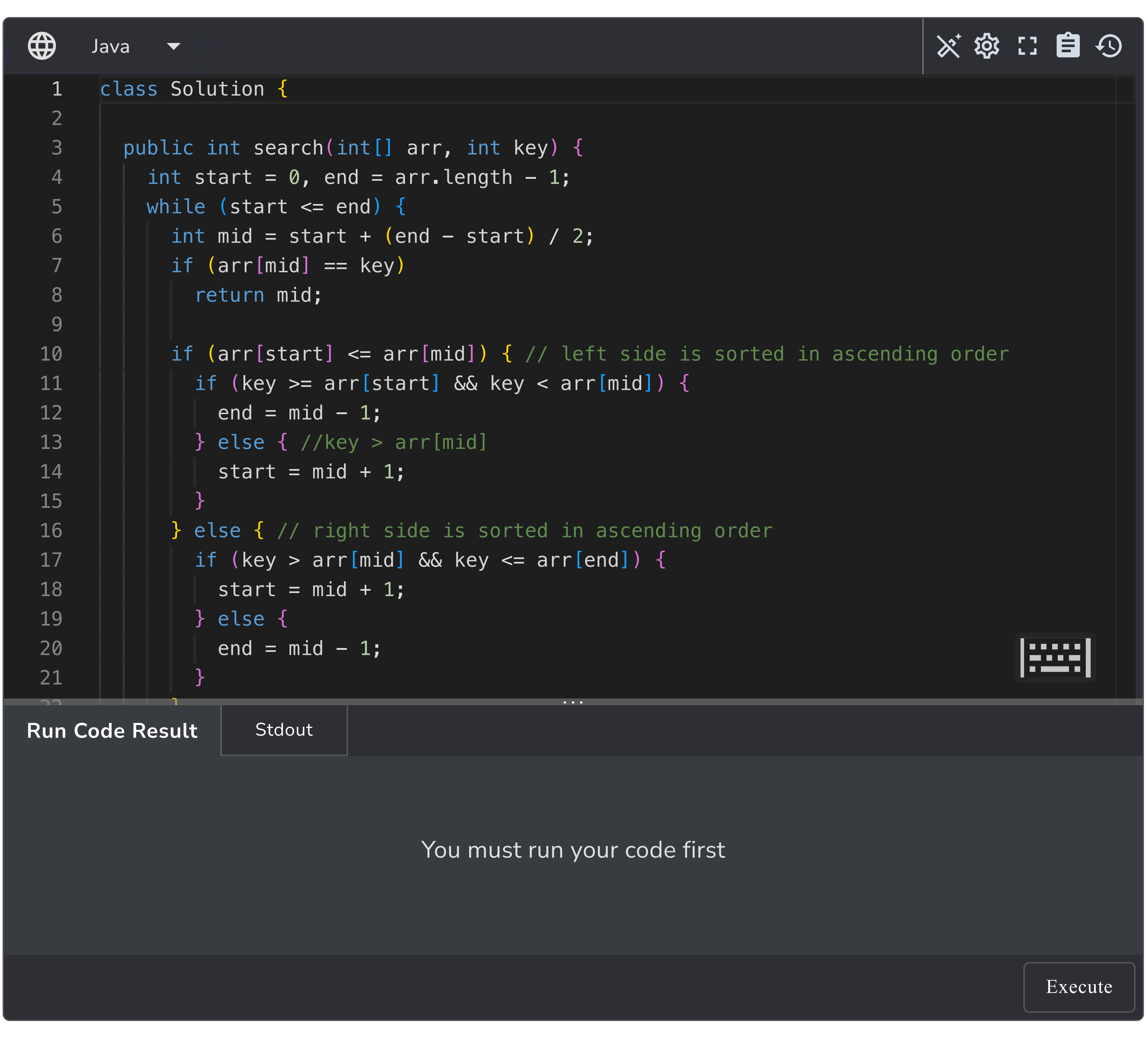
By looking at the sorted part, from its start and end, we can conclude that the key can't be in the sorted part, hence **end = middle - 1**

We have found the key!

Since there are no duplicates in the given array, it is always easy to skip one part of the array in each iteration. However, if there are duplicates, it is not always possible to know which part is sorted. We will look into this case in the 'Similar Problems' section.

Code

Here is what our algorithm will look like:



```java
class Solution {

  public int search(int[] arr, int key) {
    int start = 0, end = arr.length - 1;
    while (start <= end) {
      int mid = start + (end - start) / 2;
      if (arr[mid] == key)
        return mid;

      if (arr[start] <= arr[mid]) { // left side is sorted in ascending order
        if (key >= arr[start] && key < arr[mid]) {
          end = mid - 1;
        } else { //key > arr[mid]
          start = mid + 1;
        }
      } else { // right side is sorted in ascending order
        if (key > arr[mid] && key <= arr[end]) {
          start = mid + 1;
        } else {
          end = mid - 1;
        }
```

You must run your code first

Time Complexity

Since we are reducing the search range by half at every step, this means that the time complexity of our algorithm will be O(logN) where 'N' is the total elements in the given array.

Space Complexity

The algorithm runs in constant space O(1).

Similar Problems

Since we are reducing the search range by half at every step, this means that the time complexity of our algorithm will be O(logN) where 'N' is the total elements in the given array.

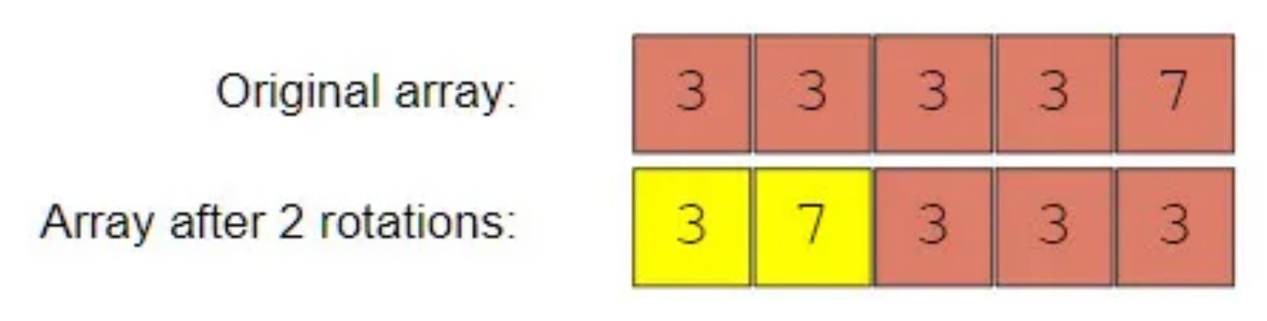
Problem 1

How do we search in a sorted and rotated array that also has duplicates?

The code above will fail in the following example!

Example 1:

```
Input: [3, 7, 3, 3, 3], key = 7
Output: 1
Explanation: '7' is present in the array at index '1'.
```

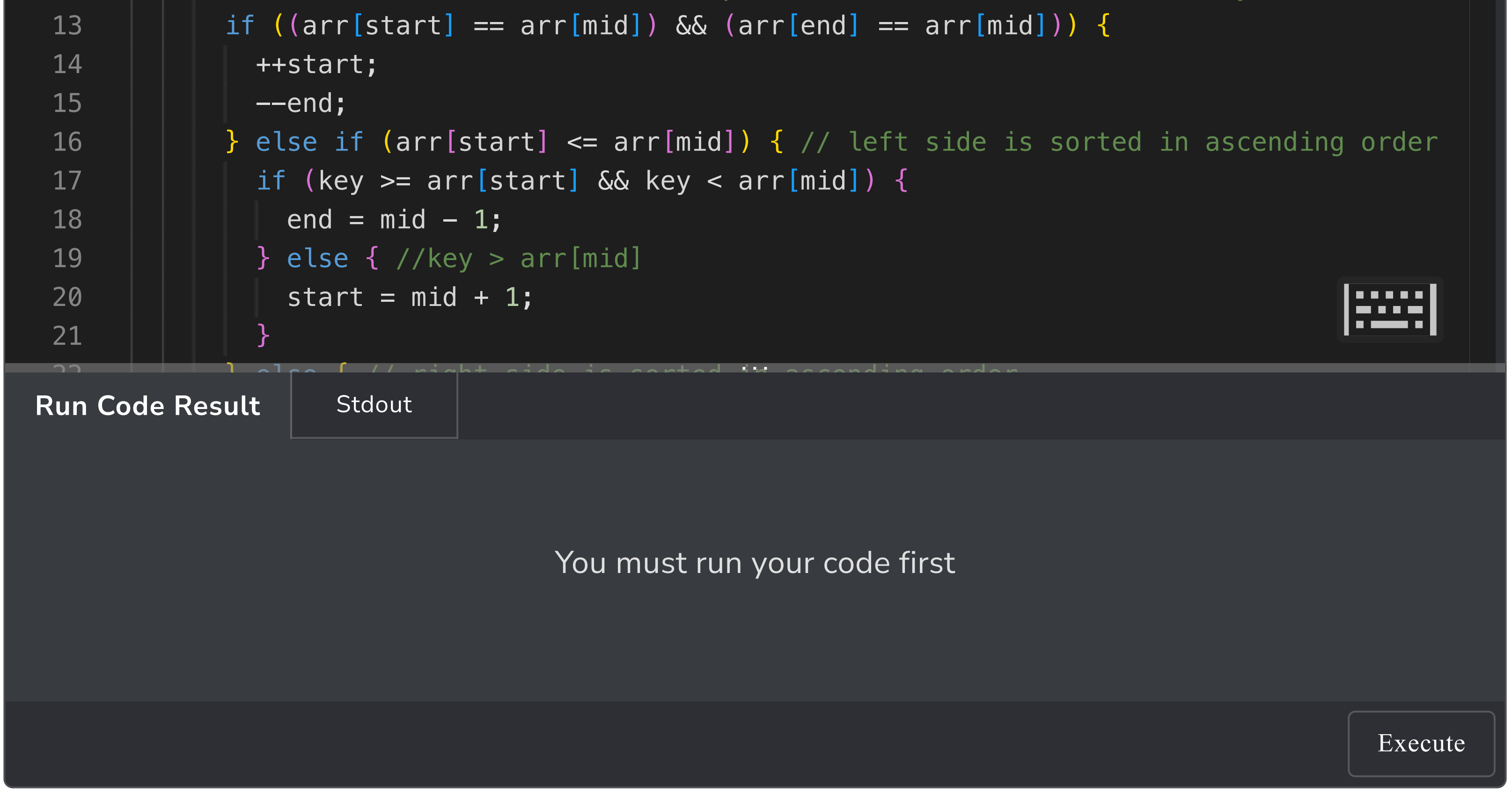


Solution

The only problematic scenario is when the numbers at indices start, middle, and end are the same, as in this case, we can't decide which part of the array is sorted. In such a case, the best we can do is to skip one number from both ends: `start = start + 1` & `end = end - 1`

Code

The code is quite similar to the above solution:



```java
class Solution {

  public  int search(int[] arr, int key) {
    int start = 0, end = arr.length - 1;
    while (start <= end) {
      int mid = start + (end - start) / 2;
      if (arr[mid] == key)
        return mid;

      // the only difference from the previous solution,
      // if numbers at indexes start, mid, and end are same, we can't choose a side
      // the best we can do, is to skip one number from both ends as key != arr[mid]
      if ((arr[start] == arr[mid]) && (arr[end] == arr[mid])) {
        ++start;
        --end;
      } else if (arr[start] <= arr[mid]) { // left side is sorted in ascending order
        if (key >= arr[start] && key < arr[mid]) {
          end = mid - 1;
        } else { //key > arr[mid]
          start = mid + 1;
        }
```

You must run your code first

Time Complexity

This algorithm will run most of the times in O(logN) . However, since we only skip two numbers in case of duplicates instead of half of the numbers, the worst case time complexity will become O(N).

Space Complexity

The algorithm runs in constant space O(1).

← Previous
Problem Challenge 2: Search In Rotated Array (Medium)

Next →
Problem Challenge 3: Rotation Count (Medium)

☐ Mark as Completed